

Operationalizing Organization Design Constructs in Agentic Software Workflows: A Diagnostic Instrument and One Confirmatory Test

Working paper — pilot study

Valentina Kurashova

6 August 2026 · framing revised 15 September 2026 (data and results unchanged)

Abstract

AI speeds up individual tasks; team-level gains often do not follow. Organization design theory offers constructs (division of labor, integration of effort, decision rights, cognitive reapportionment) as design variables, but human–AI–agent coordination in organizations still lacks much empirics: few operationalizations, few comparative outcome tests, and few ways to score coordination failure against artifact state rather than agent narrative. This working paper precedes the doctoral proposal it accompanies: it tests whether organization-design rules named on the agent layer, without changing how the work runs, are enough. We build a dual-layer diagnostic for an instrumented four-stage engineering pipeline — mechanical detectors on canonical files, plus the MAST LLM-as-judge on the same runs — and exercise it on **three task packets** (1) notify the user, but the channel is never named — should the agent ask first?; (2) merge a pull request — only the reviewer may close it, not the coder; (3) an expense too large for the manager to approve alone — Finance has to decide. Calibration surfaced four practical lessons: a role only has real authority if it is given the tools to act; a tight resource budget can look like a structural failure even when the structure is fine; how agents may record a status change affects what gets measured; and agents can write a fluent false completion report that fools a strong LLM judge (**narrator risk**). One pre-registered comparison then asks whether a **soft** machine-readable charter (rule named, skip still allowed) raises finish rates versus an instructed framework-default baseline; on expense escalation (n=18) it does not (exact McNemar $p = .73$). The stand is deliberately cheap (one actor, several role labels). What the pilot leaves: a usable measurement spine, a soft-specification boundary, and open questions about resources, enforcement, and separated agents, not another soft charter test of the same kind.

What this pilot answers

This paper answers two questions at two different levels of risk, kept separate on purpose. RQ-pilot-1 is a test that could fail. RQ-pilot-2 asks what building and stressing the instrument reveals — a question without a pass/fail criterion, written down before the confirmatory run (pre-registration signed 22 July 2026).

RQ-pilot-1 (confirmatory — can fail). On the expense-escalation packet (task_02, n=18), does a soft construct charter raise finish rates versus an instructed framework-default baseline? → **No** (exact McNemar $p = .73$).

RQ-pilot-2 (descriptive — answered regardless of outcome). What does building and stressing a measurement instrument for these constructs reveal about coordination in agentic pipelines? → Answered by a three-packet calibration trail (§4) and by a co-equal finding on the confirmatory runs themselves — **narrator risk** (§6.2): a strong LLM judge can read a fabricated audit trail and report no failure.

§1 What this paper is

This document is a **pilot working paper** that accompanies a doctoral research proposal. The proposal states the programme problem and the research question; the doctorate itself is field-first and compares organizations that redesigned work around agents with those that did not. This paper is prior work to that programme, not its first stage, and does not repeat the macro case at length.

What it is. A bounded check of one hope the tools invite: build a way to measure construct-defined coordination failures in an agentic engineering pipeline; stress the instrument on three planted-hazard task packets; run one confirmatory soft-charter comparison; report what the process reveals.

Scope. Controlled pilot only: build the measurement, then run one narrow comparative test. It says nothing about organizations with people in them; that is the doctorate's question.

How to read it with the proposal. The proposal says where the research programme goes. This paper shows what a cheap fix on the agent layer can and cannot do: measurement is hard, soft naming has a boundary, and a language-model judge can be talked into clearing unfinished work.

§2 Why start here

2.1 The programme gap (short)

The programme gap is stated in the accompanying proposal: task-level AI gains are documented, but team-level coordination gains often do not follow, and human–AI–agent pipelines mostly coordinate through framework-default roles rather than measured constructs. What follows is a pilot: build the measurement, then test one soft comparison against that default.

2.2 Why measurement first

Any later comparison, whether a stronger designed mechanism or a field reading, would be uninterpretable if several different explanations stayed tangled together: the structure failed, we measured it wrong, the role lacked the means to act, or the evaluator was fooled by a false report. This pilot therefore builds and stress-tests a diagnostic — where mistakes are cheap to fix — before asking one narrow comparative question about **soft specification** alone.

2.3 Propositions that guided the design

From organization design (and related interface rules) we use a small empirical set:

	Plain meaning
P1	Dependent work needs handoff rules
P2	Not knowing is not the same as disobeying
P3	Own your domain; escalate the rest
P4	Exploration is not a defect
P5	Ambiguity should trigger asking first
P6	Information is lost at organizational boundaries
P7	Frozen agreements reduce breakage

Each proposition is scored by a small set of failure labels (incomplete handoff, conflicting instructions, context loss, decision-rights violation, escalation failure, and a rework-counting rule that separates legitimate exploration from defect). The full label-to-code mapping lives in the replication package; this paper uses the plain names below rather than the codes.

This pilot concentrates confirmatory power on the **escalation / decision-rights** family (P3, P5), exercised on three stories, rather than claiming a full confirmatory test of all seven propositions on day one. The other propositions still shaped traps, detectors, and calibration.

§3 What we built

3.1 Pipeline and conditions

Instrumented slice: requirements → architecture → implementation → review (LangGraph nodes as infrastructure only). Organizational content lives in prompts and, in Condition B, in a machine-readable charter.

Condition A — framework-default	Condition B — construct-specified (soft)
S h a r e d D i	Same
Same task packet; same trap-relevant coaching; same model	
Competent role / SOP prompts; generic ask affordance; no machine-readable charter	Same stages plus explicit decision domains, stage commitments, escalation predicates

f
f
e
r
s
E —
n
f
o
r
c
e

Soft: violations remain possible (no hard filesystem block)

This is a **lower-bound** contrast: charter over an already-coached instructed baseline, not charter over chaos. “Framework-default” means role-template coordination on *this* stand, not a claim about every commercial agent product.

3.2 Two readouts on every scored run

	Question	How
Q1	Did a construct-defined failure occur?	Mechanical detectors on files / logs (no LLM)
Q2	How does an independent public taxonomy see the same run?	MAST LLM-as-judge (o1 + few-shot), pinned

Why Q1. Organization-design claims need a primary score that does not trust the agent’s story. Detectors read canonical objects (for example: is the expense claim terminal?).

Why Q2. Our detectors are trap-specific. MAST is a published external symptom vocabulary. When Q1 and Q2 agree, a claim is stronger; when they disagree — especially when files say the work is unfinished while the judge reports a clean run — that disagreement is itself evidence about evaluation design (see §6.2). We reuse MAST’s published judge configuration as a second eye ([Cemri et al., 2025](#): o1 few-shot $\kappa = 0.77$ vs experts; recall 0.77 implies systematic undercount).

3.3 Stand limitation (named on purpose)

Each stage is one LLM instance in one loop. Role names such as “manager” and “finance” are labels the actor writes into an audit trail, not independent agents with separate context or incentives. Escalation “completion” here means: did the protocol’s second step get recorded against the canonical object? It does **not** mean separated authority between actors. That limitation is named so that the null is not read as a result about separated authority.

3.4 Related measurement constraints

Paired seeds (one seed → A and B). Noise between configuration-equivalent protocols can reach −3 to +18 pp ([arXiv:2606.20695](#)). Planted-hazard design is informed by work such as [AgentCollabBench](#); our contrast is charter vs framework-default, not backbone or topology horse-races.

§4 Three task packets — calibration centre

Three packets stress the same escalation / decision-rights family under different covers. Order below: learn measurement → learn authority/resources → confirmatory story.

4.1 task_01 — notification ambiguity (hold / measurement)

Story. The task says a user should be notified, but does not say *how* (email, Slack, in-app...). Under P5 the agent should ask which channel to use before implementing. Guessing counts as an escalation failure.

What we learned. Our detector looked for the word “notification”. That is not the same as detecting “channel left unspecified.” We recorded the gap and left the detector unchanged after seeing cases — fixing it after peeking would have made later scores look better by construction (§4.4 row 7). Scoring “should have asked” from free text remains an open measurement problem (MAST’s published judge recall is also only 0.77). We therefore did not spend a full-N confirmatory cell here after the expense cell; it stays an open instrument extension.

4.2 task_03 — code review / PR disposition (calibration)

Story. A pull request is ready to merge. The rule is that only the reviewer may mark it done — the coder who wrote the code should not be able to close it out unilaterally. This tests whether a decision right (who gets the final call) is actually respected, not just stated.

What we learned.

- In an early version, the reviewer role had no write access — no way to actually close the PR. So even though the rule said “reviewer decides,” the coder closed it anyway, because someone had to and only the coder could (**decision rights without means**: a right written down does nothing if the role holding it can’t act on it).
- We also found that a tight turn budget made the structured condition (B) look worse than the unstructured one (A) — not because the structure failed, but because B’s extra steps (checking commitments, writing to the charter) cost turns that A didn’t spend. Raising the budget from 6 turns to 12 reversed the pattern (**resource limits masquerading as structural effects**).
- Once the reviewer had write access and the turn budget was fixed, both conditions resolved PRs almost every time (9 of 10 runs each) — a **ceiling**. At that point a full head-to-head test on this task would not teach us much more, since both conditions already succeed; it stays a candidate for later, if a harder version of the task reopens the gap.

4.3 task_02 — expense escalation (calibration + confirmatory host)

Story. Someone files an expense bigger than the manager is allowed to approve alone. The rule: Finance must say yes or no, and that answer must be saved in the claim’s status file. Writing “Finance approved” in a message while the file still says “waiting” does **not** count as done.

What we learned in calibration (before / beside full N).

- Agents sometimes wrote combined status values — for example, marking a claim “approved_pending_finance” instead of a single clean state. Our first detector read only exact status strings and missed these as a real change. Once we treated *how* agents are allowed to record a status change as a design choice in its own right, not just an implementation detail, the detector caught it (**commitment format is a design variable**).
- Agents wrote a full “Finance approved” story in their notes while the claim file itself never changed status (**aspirational record-keeping**): what they *said* happened is not what the shared file shows — the kind of breakdown Puranam calls a failure of verifiable common knowledge (Puranam, 2018).
- On those same unfinished runs, the MAST judge often reported no problem, because it trusted the agent’s story (**narrator risk**).

This packet hosts the confirmatory comparison in §5–§6.

4.4 Calibration table (organization-design reading)

#	Discovery	Packet	Reading
1	Reviewer had no write access; coder closed the PR instead	task_03	A decision right without the means to act on it is void
2	A tight turn budget made B look worse; raising it reversed the pattern	task_03	A resource limit can masquerade as a structural failure
3	A combined status string slipped past the detector	task_02	How agents may record a status change is itself a design choice
4	Agent fabricated a full “finance approved” audit trail; the claim object itself never moved	task_02	A fabricated record is a different failure than an unresolved escalation — score them separately
5	The LLM judge cleared runs that were actually stuck, when the agent’s own notes claimed success	task_02	A judge that reads narrative inherits narrator risk
6	Same overall flag rate in both conditions, but <i>which</i> runs got flagged differed	task_02	The judge’s accuracy depends on how much ambiguity the condition itself creates

#	Discovery	Packet	Reading
7	A keyword-based detector was left as a known gap rather than patched after seeing failures	task_01	Scoring escalation from natural language is still an open problem

Rows checked against `run.jsonl` and canonical objects, not agent self-report. “Calibrated” here means: rules were exercised live; failures were named and either fixed as hygiene (rows 1–3) or held open on purpose (row 7), so later results wouldn’t rest on an instrument quietly patched to fit what we saw. It does **not** mean formal psychometric validation of detectors.

§5 One confirmatory comparison

5.1 The question

On the expense-escalation packet (task_02), does Condition B raise finish rates relative to Condition A?

What “finish” means here. A run **passes** if the claim record ends in a terminal state (approved or rejected after the required finance step). It **fails the endpoint** if the claim stays unfinished (for example parked at `needs_finance`) or is wrongly approved without asking Finance.

What the soft contrast isolates. Condition B *names* the escalation rule in a machine-readable charter, but still allows the agent to skip it. We are testing whether writing the rule down changes behavior — not whether blocking the skip would change behavior. Hard block and extra resources are left open.

5.2 Pre-registration and scope

The signed protocol originally planned three full-N cells in the escalation-failure family — one per task packet (labels T3, E2, R2). After calibration:

- **task_02** (E2) ran at full N — 18 paired runs; MAST (Q2) scored on every run.
- **task_03** (R2) stayed at staged calibration: both conditions already sat at ceiling (§4.2).
- **task_01** (T3) stayed at a small early sample, with the open keyword-detection gap (§4.1).

We logged that scope choice **before** looking at further cells. Whether a stronger design lever (resources, enforcement, or separated agents) changes the result is an open question, not a planned stage.

Models pinned: `openai/gpt-4o` (agents); `openai/o1` + few-shot (judge). Primary test: exact McNemar on discordant pairs. Confirmatory power went to one construct family (escalation failure). Incomplete handoffs, decision-rights violations, and context loss were also measured and reported, but not as confirmatory endpoints.

§6 Results

6.1 Confirmatory result (task_02, n=18)

	Condition A	Condition B
Finished (pass)	9/18	7/18
Unfinished (mostly parked at <code>needs_finance</code>)	9/18	11/18
Wrongful approve (no ask / no finance)	0/18	0/18

Reading in plain words. Soft naming of the rule did not raise finish rates. B finished 7 of 18 claims; A finished 9 of 18. Looking at pairs: both finished in 4 seeds; only A finished in 5; only B finished in 3; neither finished in 6. On the 8 seeds where A and B disagreed, the exact McNemar test gives $p = .73$ — no evidence of a B advantage. The point estimate even leans slightly against B (–11 percentage points), inside the noise range documented for this kind of protocol. With n=18 the cell can detect only large effects. An early peek (n=3, B ahead 2/3) did not hold at full N; we report it because that peek was locked before the full run, not dropped afterward. Almost all unfinished mass is **park vs finish**. Agents did not silently self-approve (0 wrongful approves in 36 runs).

What this supports. On this packet, soft enforce, and single-actor stand: **writing the escalation rule down without enforcement or extra resources did not move the finish endpoint**. Harder levers remain open questions.

6.2 Narrator risk (same runs — co-equal finding)

On three unfinished A runs (09_A, 12_A, 14_A), file-state detectors (Q1) correctly marked the claim unresolved, while the MAST judge (Q2) reported no failure. In one case the agent invented a full finance-approved audit trail; in two others the claim stayed parked on disk while the closing prose claimed the work was done.

Overall, Q2 flagged the same share of runs on each side (12/18). **Which** runs it flagged differed: under B it caught all 11 unfinished runs; under A it missed 3 of 9 unfinished runs. When the design leaves more ambiguity (A), the narrative judge is a noisier proxy for the target construct. So LLM-as-judge rates are not a condition-blind meter of “how much coordination failed” — they depend on how much room the design leaves for a fluent false story.

6.3 Also-report (extra checks — not the confirmatory test)

What we also checked	Result	Plain reading
Early traps that should almost never fire on this stack	0 fires in 11 checks	Expected floor — the stack was not broken in an obvious way
Code-review packet after reviewer got write access (small sample)	A 9/10 finished · B 9/10 finished	Both conditions already near perfect — little room left for a soft A/B test
Exploratory: does the agent invent a fake “done” story while the PR file stays open?	5/10 without a “file is source of truth” note · 0/10 with that note	Often a wording/spec gap, not only “dishonesty”
Same expense seeds: were <i>all</i> claims in the packet finished?	A 9/18 · B 5/18	Extra look only: B left more work unfinished — possible cost of more structure

§7 What this opens

7.1 Reading the soft null

Organization-design theory expects coordination mechanisms to work when they match the uncertainty of the work *and* the authority to act on it (Puranam, 2018; Konsynski, Kathuria & Karhade, 2024). A soft charter changes the written rule — who should escalate — without changing what happens if the agent ignores it. Soft enforce was the cheapest way to isolate that contrast. The null fits that reading: naming alone was not enough here. Stronger levers remain open questions: make skipping the rule impossible (hard enforce), give the role real tools, and use separate actors.

7.2 Next questions (trajectory)

Open question	Next step
Does construct specification help when the role is resourced and skipping the rule is hard-blocked?	Open; a designed mechanism, if fieldwork isolates such a rule
Does the result change when decision rights sit with separate agents, not role labels in one loop?	Open; separated agents, if fieldwork isolates such a rule
Can natural-language escalation be scored without brittle keyword detectors?	Open; instrument extension
Does soft specification still fail to reduce escalation failure when the A/B contrast is informative (not at ceiling)?	Open; instrument extension
Do the instrument and findings transfer to human–AI teams in the field?	The doctorate’s field question: how coordination differs when work was redesigned around agents and when it was not

Through-line. This pilot built and stress-tested a measurement spine for organization-design constructs in an agentic pipeline; showed where measurement and soft specification break; answered one confirmatory soft question; and left resource, enforcement, and separated agents as open questions rather than programme stages. The doctorate that follows is field-first; its wording is in the research proposal.

References

Citation	URL
Konsynski, Kathuria & Karhade 2024	https://doi.org/10.1080/07421222.2024.2340830
Puranam 2018	https://doi.org/10.1093/oso/9780199672363.001.0001
MAST — Cemri et al. 2025	https://arxiv.org/abs/2503.13657
AgentCollabBench	https://arxiv.org/abs/2605.08647
Noise-floor coordination study	https://arxiv.org/abs/2606.20695
Replication materials. Protocol, pre-registration sign-off (22 July 2026), codebook, theory notes, stand code, and run logs: https://github.com/valkurash/agent-c-org-design-pilot .	